InterviewAce

Requirements Specification

AI-Powered Interview Practice Platform

Course	CS 4800
Team	Freeman Yiu (Frontend & AI Systems) · Youcheng Taing (Backend & ML)
Stack	React · TypeScript · FastAPI · PostgreSQL · sentence-transformers · Ollama · Docker
Repository	github.com/Youcheng9/Job-Interview-Prep-Tool (branch: eval)

Scope: This document defines the functional and non-functional requirements for InterviewAce, an AI-powered interview preparation web platform. Functional requirements specify what the system does. Non-functional requirements define quality attributes — each includes a measurable, objectively testable verification criterion.

Section 1 — Functional Requirements

FR-01 User Registration & Authentication

The system shall allow users to register with an email and password, log in securely, and maintain authenticated sessions using JWT tokens. All protected API endpoints shall reject unauthenticated requests with HTTP 401.

FR-02 Password Reset via Email

The system shall support a forgot-password flow that generates a time-limited, single-use reset token, emails the reset link to the user, and allows the user to set a new password by providing the valid token.

FR-03 Role & Level Selection

The system shall present users with selectable job-role tracks (SWE, Data Science, PM, Behavioral) and candidate levels (Intern, New Grad). Selecting a role and level shall load the matching curated question bank.

FR-04	**Question Delivery with Filters**

The system shall retrieve and display role-specific and level-specific interview questions one at a time, supporting filters by topic, difficulty (easy / medium / hard), and completion status (done / not done). Question order shall be shuffled and persisted across sessions.

FR-05	**Answer Submission (Text & Speech)**

The system shall accept free-text answers (minimum 20 characters) typed by the user or dictated via browser Speech-to-Text (Web Speech API). Each submission shall be persisted to the database with a timestamp and user reference.

FR-06	**AI-Powered Answer Scoring**

The system shall embed submitted answers using sentence-transformers (all-MiniLM-L6-v2) and compute cosine similarity scores against predefined rubric components for each question. Behavioral answers shall additionally be evaluated using STAR-framework signal detection (situation, task, action, result, reflection).

FR-07	**Multi-Dimensional Score Display**

The system shall calculate and display four independent dimension scores (Technical Depth, Clarity, Completeness, Structure) on a scale of 0–100 for every submitted answer, along with an overall weighted score.

FR-08	**Instant Deterministic Feedback**

The system shall generate actionable improvement suggestions by identifying missing rubric concepts (those scoring below the 0.52 threshold) and surfacing them as specific, non-generic tips. Feedback shall be deterministic: identical inputs shall always produce identical feedback output.

FR-09	**AI Coach Chat (Ollama / llama3.2)**

After viewing a score, the user shall be able to open an AI Coach chat panel for that answer. The system shall maintain a per-answer conversation thread in the database and forward messages to a locally-hosted Ollama model (llama3.2:3b) to generate coaching replies.

FR-10	**Answer History & Progress Tracking**

The system shall store all past answers and scores per user and display a chronological history page showing per-question score trends over time, filterable by role and level.

FR-11	Horizontal Bar Chart Score Visualization
	The system shall render a horizontal bar chart (via Recharts) displaying the user's four scoring dimensions — Technical Depth, Clarity, Completeness, and Structure — as individual labelled bars on a 0–100 scale. Each bar shall be colour-coded by score range and update immediately upon receiving a new scored submission.

FR-12	Question Bank Management
	The system shall support the ability to add, update, and manage role-specific and level-specific questions together with their associated rubric components (ideal answer, concept list, dimension keywords) via a data-layer seeding interface.

Section 2 — Non-Functional Requirements

Each non-functional requirement below includes a **Verification Criteria** row (highlighted in teal) that defines a measurable, objectively testable acceptance criterion.

NFR-01	AI Scoring Response Time
	The system shall return multi-dimensional scores and instant feedback to the user within 3 seconds of answer submission under normal load (up to 50 concurrent users).
Verification Criteria	Load test with 50 simulated concurrent users submitting answers via POST /scoring/submit. Measure p95 response time from request to score display. Pass criterion: p95 <= 3,000 ms with zero 5xx errors.

NFR-02	Embedding Cache Latency Reduction
	The system shall reduce redundant embedding computation time by at least 70% for repeated rubric lookups when the in-memory LRU cache (lru_cache on _cached_embeddings) is active.
Verification Criteria	Run 100 identical rubric embedding requests with cache cold (process restart) to establish a baseline average latency, then re-run with cache warm. Pass criterion: warm-cache average latency reduction >= 70% vs cold baseline.

NFR-03	**Page Load Time (LCP)**
	All primary UI pages (role selection, Q&A interface, score dashboard, history page) shall achieve a Largest Contentful Paint (LCP) of under 2.5 seconds on a standard broadband connection.
Verification Criteria	Run Lighthouse audit on each primary page three times from a clean browser profile on a standard broadband connection. Pass criterion: LCP <= 2,500 ms on all three runs for each page.

NFR-04	**Task Completion Without Assistance**
	A first-time user shall be able to select a role, answer a question, and view their score without any external guidance or documentation.
Verification Criteria	Conduct a moderated usability test with 5 new users. Each user attempts: (1) select a role, (2) answer a question, (3) view the score card. Observe without intervention. Pass criterion: >= 4 out of 5 users complete all three tasks without requesting help.

NFR-05	**Mobile Responsiveness**
	The platform shall be fully functional and readable on screen widths from 375 px (iPhone SE) to 1440 px (desktop), with no horizontal scrolling or broken layouts at any breakpoint.
Verification Criteria	Test all primary pages in Chrome DevTools responsive mode at widths 375 px, 768 px, and 1440 px. Pass criterion: no horizontal scroll bar visible, all content legible and interactive, no overlapping elements at any tested width.

NFR-06	**System Uptime**
	The deployed platform on Render shall maintain >= 99% uptime during the 9-week evaluation period, excluding scheduled maintenance windows.
Verification Criteria	Monitor availability using an external uptime service (e.g., UptimeRobot) with 1-minute polling for the full 9-week period. Pass criterion: total recorded downtime <= 1.08 hours (1% of 108 hours) excluding announced maintenance.

NFR-07	**JWT Authentication Enforcement**
	All protected API endpoints shall reject requests with a missing, expired, or tampered JWT token with HTTP 401 Unauthorized. Tokens shall expire after 24 hours.
Verification Criteria	Send 20 API requests to each protected endpoint under three conditions: no token, expired token, and valid token. Pass criterion: all 40 unauthorized requests return HTTP 401; all 20 valid requests return HTTP 200.

NFR-08	Password Hashing with bcrypt
	User passwords shall never be stored in plaintext. All passwords shall be hashed using bcrypt with a minimum cost factor of 12 before database persistence.
Verification Criteria	Register a test user, then inspect the stored password_hash column in the users table directly via database query. Pass criterion: stored value begins with '$2b$12$' (bcrypt cost 12) and is not equal to the original plaintext password.

NFR-09	CI Pipeline Execution
	All code merged to the main branch shall automatically pass a GitHub Actions CI pipeline that runs unit tests, lint checks, and a Docker build within 5 minutes.
Verification Criteria	Merge a pull request to main and observe the GitHub Actions workflow run. Pass criterion: pipeline completes in <= 5 minutes with all checks (pytest, lint, docker build) reporting success on a clean codebase.

NFR-10	Database Schema Migrations
	All changes to the database schema shall be managed through versioned Alembic migration scripts, allowing the schema to be reproduced from scratch via migration history alone.
Verification Criteria	Drop the database on a clean machine, then run 'alembic upgrade head' from the repository root. Pass criterion: all 8 migration versions apply without errors and the resulting schema is identical to the production schema with no manual SQL required.

NFR-11	Single-Command Docker Deployment
	The full application stack (frontend, backend, PostgreSQL database) shall be launchable on any host using a single 'docker-compose up' command without manual environment configuration beyond a .env file.
Verification Criteria	Clone the repository onto a clean machine, provide only the .env file, run 'docker-compose up --build'. Pass criterion: all services start and the application is fully functional within 3 minutes with no additional configuration steps.

NFR-12	Answer Persistence on Restart
	Every answer submitted by a user shall be durably stored in PostgreSQL. No submitted answer shall be lost in the event of an application server restart or container crash.
Verification Criteria	Submit 20 answers, then simulate an application container restart using 'docker restart <backend-container>'. Query the answers table after restart. Pass criterion: all 20 answers are retrievable and correctly associated with the user_id and question_id.

NFR-13	Deterministic Scoring Consistency
	The AI scoring engine shall produce identical scores for identical answer–rubric pairs across repeated invocations, ensuring deterministic output with no randomness in the scoring pipeline.
Verification Criteria	Submit the same answer text to the same question 10 times in sequence via POST /scoring/submit. Pass criterion: all 10 returned score objects are bit-for-bit identical across all four dimensions (technical_depth, clarity, completeness, structure) and overall score.

Section 3 — Requirements Summary

Functional Requirements

ID	Title	Type
FR-01	User Registration & Authentication	Functional
FR-02	Password Reset via Email	Functional
FR-03	Role & Level Selection	Functional
FR-04	Question Delivery with Filters	Functional
FR-05	Answer Submission (Text & Speech)	Functional
FR-06	AI-Powered Answer Scoring	Functional
FR-07	Multi-Dimensional Score Display	Functional
FR-08	Instant Deterministic Feedback	Functional
FR-09	AI Coach Chat (Ollama / llama3.2)	Functional
FR-10	Answer History & Progress Tracking	Functional
FR-11	Horizontal Bar Chart Score Visualization	Functional
FR-12	Question Bank Management	Functional

Non-Functional Requirements

ID	Title	Category	Type
NFR-01	AI Scoring Response Time	Performance	Non-Functional
NFR-02	Embedding Cache Latency Reduction	Performance	Non-Functional
NFR-03	Page Load Time (LCP)	Performance	Non-Functional
NFR-04	Task Completion Without Assistance	Usability	Non-Functional
NFR-05	Mobile Responsiveness	Usability	Non-Functional
NFR-06	System Uptime	Reliability	Non-Functional
NFR-07	JWT Authentication Enforcement	Security	Non-Functional
NFR-08	Password Hashing with bcrypt	Security	Non-Functional
NFR-09	CI Pipeline Execution	Maintainability	Non-Functional
NFR-10	Database Schema Migrations	Maintainability	Non-Functional
NFR-11	Single-Command Docker Deployment	Scalability	Non-Functional
NFR-12	Answer Persistence on Restart	Data Integrity	Non-Functional
NFR-13	Deterministic Scoring Consistency	Scoring Accuracy	Non-Functional

Total: 12 Functional Requirements + 13 Non-Functional Requirements = 25 Requirements